Conectar PokeAPI a tu app de clima

Solo se cambia la API. Las entidades, el repositorio, el ViewModel y las pantallas **quedan igual**. El Pokémon se muestra con el mismo formato que ya tienes.

La idea en una página

Tu app está separada en capas. La pantalla no sabe de dónde vienen los datos: solo conoce las **entidades** (`RespuestaClima`, `Ubicacion`, `ClimaActual`...). Por eso puedes cambiar la API sin tocar la pantalla. Basta con que la capa de datos siga entregando **las mismas entidades**.

```
Buscador → ClimaRepository → WeatherApiService → ApiClient → Internet (PokeAPI)
Pantalla ← Entidades (iguales) ← Mapper ← DTO ← JSON del Pokémon
```

Los archivos en **negrita** son los únicos que cambian:

#	Archivo	Qué decide	Qué cambias
1	`apikey.properties`	La clave	Un valor cualquiera (no es código)
2	`datos/ApiClient.kt`	**A qué servidor** te conectas	1 línea: la URL base
3	`datos/WeatherApiService.kt`	**Qué ruta** pides y cómo mandas el nombre	3 líneas
4	`datos/dto/ClimaDtos.kt`	**Qué forma** tiene el JSON	Todo el archivo
5	`datos/mapper/ClimaMapper.kt`	**Dónde se muestra** cada dato	Todo el archivo

> **Respaldo**
> En la carpeta del proyecto ya tienes `respaldo_weatherapi/` con los 4 archivos originales. Si algo sale mal, copia el original encima y vuelves a como estaba.

Paso 0 · Conocer la API en el navegador

Antes de programar, mira siempre qué devuelve la API. Abre en el navegador:

```
https://pokeapi.co/api/v2/pokemon/ditto
```

Vas a ver un JSON largo. Solo usaremos estos campos:

Campo del JSON	Valor en Ditto	Qué es
`id`	132	Número de Pokédex
`name`	"ditto"	Nombre (en minúsculas)
`height`	3	Altura en **decímetros** (÷ 10 = metros)
`weight`	40	Peso en **hectogramos** (÷ 10 = kg)
`base_experience`	101	Experiencia base
`sprites.front_default`	URL .png	Imagen de frente
`sprites.back_default`	URL .png	Imagen de espalda
`sprites.front_shiny`	URL .png	Imagen "shiny"
`types[].type.name`	"normal"	Tipos (lista)
`abilities[].ability.name`	"limber", "imposter"	Habilidades (lista)
`stats[].base_stat` + `stats[].stat.name`	48 + "hp"	6 estadísticas: hp, attack, defense, special-attack, special-defense, speed

Ahora prueba esta otra URL. Tiene los mismos parámetros que tu app ya manda a WeatherAPI:

```
https://pokeapi.co/api/v2/pokemon/ditto?key=x&days=3&lang=es&aqi=no&alerts=no
```

> **Qué aprendes aquí**
> Si la segunda URL también muestra a Ditto, PokeAPI **ignora** los parámetros que no conoce. Por eso no hace falta quitar los `@Query` del servicio: siguen viajando, pero PokeAPI no los usa.
> Si la segunda URL te diera error, avísame: habría que quitarlos.

Cómo se arma la URL

```
https://pokeapi.co/api/v2/   pokemon/   ditto
└──────── URL base ─────────┘  └ ruta ┘  └ lo que escribe el usuario ┘
      (ApiClient.kt)          (@GET en WeatherApiService.kt)
```

La diferencia importante: en WeatherAPI la ciudad iba **después del** `?` como parámetro (`?q=La Paz`), y eso en Retrofit es `@Query` . En PokeAPI el nombre va **dentro de la ruta** (`/pokemon/ditto`), y eso en Retrofit es `@Path` .

Paso 1 · La clave (sin tocar código)

PokeAPI **no usa clave**, pero tu código sí la exige:

- `ClimaRepository.kt` (línea 17) muestra "Falta la clave..." si la clave está vacía.
- `ClimaViewModel.kt` (línea 28) activa el **modo demo** si la clave está vacía.

Como no queremos tocar esos archivos, le damos una clave cualquiera. En la **raíz del proyecto** (junto a `apikey.properties.example`) crea el archivo `apikey.properties` con:

`apikey.properties`

```
WEATHER_API_KEY=pokeapi
```

- ☐ Guarda el archivo.
- ☐ En Android Studio pulsa **File** → **Sync Project with Gradle Files** (el ícono del elefante).

> **Qué aprendes aquí**
> `build.gradle.kts` lee ese archivo y crea `BuildConfig.WEATHER_API_KEY`. El valor da igual porque PokeAPI lo ignora. Solo sirve para que el código no lo vea vacío.

Paso 2 · La URL base (ApiClient.kt)

Abre `app/src/main/java/com/example/clima/datos/ApiClient.kt`. Cambia **solo esta línea**:

antes

```
.baseUrl("https://api.weatherapi.com/v1/")
```

después

```
.baseUrl("https://pokeapi.co/api/v2/")
```

> **Regla de Retrofit**
> La URL base **siempre termina en** `/`. Sin ella, la app se cierra al arrancar con `baseUrl must end in /`.

> **Qué aprendes aquí**
> `ApiClient` es "el teléfono": decide a qué servidor llamas y con qué tiempos de espera (`connectTimeout`, etc.). Todo lo demás de este archivo sirve igual para cualquier API.

Paso 3 · La ruta y el nombre (WeatherApiService.kt)

Abre `datos/WeatherApiService.kt`. Vas a cambiar **3 cosas**.

3.1 · La ruta del `@GET`

antes

```
@GET("forecast.json")
```

después

```
@GET("pokemon/{nombre}")
```

Las llaves `{nombre}` son un **hueco**: Retrofit lo rellena con lo que escriba el usuario.

3.2 · Quién rellena el hueco: `@Query` → `@Path`

antes

```
@Query("q") ciudad: String,
```

después

```
@Path("nombre") ciudad: String,
```

El texto dentro de `@Path("nombre")` debe ser **igual** al que está entre llaves en el `@GET`. La variable sigue llamándose `ciudad` a propósito: así `ClimaRepository` no se entera del cambio.

3.3 · El import de `@Path`

Arriba, junto a los otros imports, agrega:

```
import retrofit2.http.Path
```

Cómo te debe quedar el archivo completo

datos/WeatherApiService.kt

```
package com.example.clima.datos

import com.example.clima.datos.dto.RespuestaClimaDto
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Query

interface WeatherApiService {
    @GET("pokemon/{nombre}")
    suspend fun obtenerPronostico(
        @Query("key") key: String,
        @Path("nombre") ciudad: String,
        @Query("days") dias: Int = 3,
        @Query("lang") idioma: String = "es",
        @Query("aqi") calidadAire: String = "no",
        @Query("alerts") alertas: String = "no"
    ): RespuestaClimaDto
}
```

Qué aprendes aquí

`@Query("x")` → agrega `?x=valor` al final de la URL.
`@Path("x")` → reemplaza `{x}` dentro de la ruta.
`@GET` → el tipo de petición (leer datos). También existen `@POST`, `@PUT` y `@DELETE` para enviar datos.
El nombre de la función (`obtenerPronostico`) y lo que devuelve (`RespuestaClimaDto`) no cambian, porque `ClimaRepository` los usa.

Paso 4 · La forma del JSON (dto/ClimaDtos.kt)

Un **DTO** es una clase que copia la forma del JSON. Gson lee el JSON y llena el DTO campo por campo, usando el nombre que pones en `@SerializedName`.

Abre `datos/dto/ClimaDtos.kt`, **borra todo su contenido** y escribe esto:

datos/dto/ClimaDtos.kt

```
package com.example.clima.datos.dto

import com.google.gson.annotations.SerializedName

// Modelos de los datos que llegan desde PokeAPI.

// PokeAPI repite mucho este formato: { "name": "...", "url": "..." }
data class RecursoDto(
    @SerializedName("name") val nombre: String
)

// Cada elemento de "types": { "slot": 1, "type": { "name": "normal" } }
data class TipoDto(
    @SerializedName("type") val tipo: RecursoDto
)

// Cada elemento de "abilities": { "ability": { "name": "limber" } }
data class HabilidadDto(
    @SerializedName("ability") val habilidad: RecursoDto
)

// Cada elemento de "stats": { "base_stat": 48, "stat": { "name": "hp" } }
data class EstadisticaDto(
    @SerializedName("base_stat") val valor: Int,
    @SerializedName("stat") val stat: RecursoDto
)

data class SpritesDto(
    @SerializedName("front_default") val frente: String?,
    @SerializedName("back_default") val espalda: String?,
    @SerializedName("front_shiny") val shiny: String?
)

// La clase principal conserva el nombre RespuestaClimaDto
// para que WeatherApiService y ClimaRepository no cambien.
data class RespuestaClimaDto(
    @SerializedName("id") val id: Int,
    @SerializedName("name") val nombre: String,
    @SerializedName("height") val altura: Int,
    @SerializedName("weight") val peso: Int,
    @SerializedName("base_experience") val experiencia: Int?,
    @SerializedName("sprites") val sprites: SpritesDto,
    @SerializedName("types") val tipos: List<TipoDto>,
    @SerializedName("abilities") val habilidades: List<HabilidadDto>,
    @SerializedName("stats") val estadisticas: List<EstadisticaDto>
)
```

Qué aprendes aquí

• **Objeto en el JSON** (`{ }`) → una `data class` .
• **Lista en el JSON** (`[ ]`) → `List<...>` .
• **Objeto dentro de otro** → una clase que usa otra clase (`TipoDto` contiene un `RecursoDto`).
• **Campo que puede faltar o venir** `null` → tipo con `?` (`String?`). Algunos Pokémon no tienen imagen de espalda o shiny.
• No hace falta declarar **todos** los campos del JSON: Gson ignora los que no pongas.

Es normal ver errores en rojo ahora

Después de este paso, `ClimaMapper.kt` se pondrá en rojo porque usa las clases viejas (`UbicacionDto` , etc.). Se arregla en el Paso 5.

Paso 5 · Acomodar el Pokémon en tu formato (mapper/ClimaMapper.kt)

Este es el corazón del cambio. El mapper recibe el DTO del Pokémon y devuelve **las mismas entidades de clima** que tu pantalla ya sabe mostrar. Tú decides en qué "casillero" va cada dato.

Dónde va cada dato en la pantalla

Lo que muestra la pantalla	Campo de la entidad	Dato del Pokémon
Título grande	`location.name`	Nombre ("Ditto")
Subtítulo	`location.region` · `country`	"Tipo: normal" · "Pokémon #132"
TEMPERATURA ACTUAL	`current.temp_c`	HP
Ícono del clima	`current.condition.icon`	Imagen de frente
Texto bajo la temperatura	`current.condition.text`	"Ditto · normal"
Sensación térmica	`current.feelslike_c`	Ataque
Humedad	`current.humidity`	Defensa
Viento	`current.wind_kph`	Velocidad
Precipitación	`current.precip_mm`	Peso en kg
Índice UV	`current.uv`	Altura en m
Pestaña Pronóstico (3 tarjetas)	`forecast.forecastday`	Frente / espalda / shiny, cada una con 2 estadísticas
Detalles: Latitud / Longitud	`location.lat` / `lon`	Altura / peso
Detalles: Hora local	`location.localtime`	Experiencia base
Detalles: Última medición	`current.last_updated`	Habilidades

Las etiquetas de la pantalla (por ejemplo "Humedad") **no cambian**, porque no tocamos la UI. Si quieres, puedes armar tu propia tabla y poner los datos en otros casilleros.

Abre `datos/mapper/ClimaMapper.kt`, **borra todo su contenido** y escribe:

`datos/mapper/ClimaMapper.kt`

```
package com.example.clima.datos.mapper

import com.example.clima.datos.dto.*
import com.example.clima.entidad.*

// PokeAPI no trae clima: cada dato del Pokémon se acomoda en un campo de las entidades.

// Busca una estadística por su nombre ("hp", "attack"...). Si no existe, devuelve 0.
private fun RespuestaClimaDto.stat(nombre: String): Double =
    (estadisticas.firstOrNull { it.stat.nombre == nombre }?.valor ?: 0).toDouble()

// Arma una tarjeta de la pestaña "Pronóstico" con una imagen del Pokémon.
```

```
private fun RespuestaClimaDto.tarjeta(titulo: String, imagen: String?, max: Double, min: Double) =
    DiaPronostico(
        date = titulo,
        day = ResumenDia(
            maxtemp_c = max,
            mintemp_c = min,
            daily_chance_of_rain = 0,
            condition = Condicion(text = titulo, icon = imagen.orEmpty(), code = id)
        )
    )

// Mantiene el nombre aEntidad() porque ClimaRepository la llama así.
fun RespuestaClimaDto.aEntidad(): RespuestaClima {
    val nombreBonito = nombre.replaceFirstChar { it.uppercase() }        // "ditto" → "Ditto"
    val textoTipos = tipos.joinToString(", ") { it.tipo.nombre }
    val textoHabilidades = habilidades.joinToString(", ") { it.habilidad.nombre }

    return RespuestaClima(
        location = Ubicacion(
            name = nombreBonito,
            region = "Tipo: $textoTipos",
            country = "Pokémon #$id",
            localtime = "Experiencia base: ${experiencia ?: 0}",
            lat = altura / 10.0,                                          // metros
            lon = peso / 10.0                                             // kg
        ),
        current = ClimaActual(
            temp_c = stat("hp"),
            feelslike_c = stat("attack"),
            humidity = stat("defense").toInt(),
            wind_kph = stat("speed"),
            precip_mm = peso / 10.0,
            uv = altura / 10.0,
            last_updated = "Habilidades: $textoHabilidades",
            condition = Condicion(
                text = "$nombreBonito · $textoTipos",
                icon = sprites.frente.orEmpty(),
                code = id
            )
        ),
        forecast = Pronostico(forecastday = listOf(
            tarjeta("Vista de frente", sprites.frente, stat("attack"), stat("defense")),
            tarjeta("Vista de espalda", sprites.espalda, stat("special-attack"), stat("special-
defense")),
            tarjeta("Versión shiny", sprites.shiny, stat("hp"), stat("speed"))
        ))
    )
}
```

Qué aprendes aquí

• `firstOrNull { ... }` busca el primer elemento de la lista que cumple la condición. Si ninguno la cumple, da `null`.

• `?: 0` (el operador "Elvis") significa "si es null, usa 0".

• `joinToString(", ")` une una lista en un solo texto: `["limber", "imposter"]` → `"limber, imposter"`.

• `.orEmpty()` convierte un `String?` null en `""`. Si la imagen falta, tu pantalla muestra un emoji en su lugar (`ClimaApp.kt`, línea 214).

• El nombre de la tarjeta va en `date`. Tu pantalla intenta leerlo como fecha y, si no puede, lo muestra tal cual (`ClimaApp.kt`, línea 225).

Paso 6 · Compilar y probar

☐ **Build → Rebuild Project.** No debe quedar nada en rojo.

☐ Ejecuta la app (▶).

☐ **Apaga el interruptor "Modo demostración".**

☐ Al abrir, la app busca "La Paz, Bolivia" y mostrará un error. Es normal: no existe un Pokémon con ese nombre.

☐ Borra el buscador, escribe `ditto` y pulsa **Buscar**.

☐ Revisa las 3 pestañas: **Actual** (imagen y estadísticas), **Pronóstico** (frente, espalda y shiny) y **Detalles**.

☐ Prueba `pikachu` y `25`. Pikachu tiene estadísticas distintas entre sí, así que se nota mejor qué dato va en cada casillero (Ditto tiene todas en 48).

Lo que verás (y por qué)

Situación	Qué pasa	Por qué
Escribes `Ditto` (con mayúscula)	Error HTTP 404	PokeAPI solo acepta minúsculas
Escribes un Pokémon que no existe	"El servicio no pudo responder (HTTP 404)"	`ClimaRepository` solo conoce los códigos de error de WeatherAPI
El pie de página dice "Datos: WeatherAPI.com"	Es normal	Ese texto está en la UI, que no tocamos
"Probabilidad de lluvia: 0%"	Es normal	No hay ningún dato del Pokémon para ese casillero

Problemas comunes

Síntoma	Causa	Solución
La app se cierra al abrir: *baseUrl must end in /*	Falta la `/` final	Paso 2
"Falta la clave de WeatherAPI..."	`apikey.properties` no existe o no hiciste Sync	Paso 1
Siempre muestra La Paz y no busca otra cosa	El modo demo está encendido	Apaga el interruptor
URL "pokemon/{nombre}" does not contain "{ciudad}" o parecido	El texto de `@Path` no coincide con el de las llaves	Ambos deben decir `nombre`
Unresolved reference: Path	Falta el import	Paso 3.3
Unresolved reference: UbicacionDto (u otro DTO viejo)	El mapper aún tiene código viejo	Borra todo el mapper y pega el del Paso 5
"...datos que no se pudieron interpretar"	Algún `@SerializedName` está mal escrito	Compáralo con el JSON del Paso 0
No aparecen las imágenes	Sin Internet o el sprite es null	Revisa la conexión y prueba otro Pokémon

Extra opcional · Aceptar mayúsculas y espacios

Si quieres que `Ditto` o `Mr Mime` funcionen, sin tocar nada fuera de la capa de la API, puedes agregar un **interceptor** en `ApiClient.kt`. Un interceptor es un código que **modifica cada petición antes de que salga**. Va dentro de `OkHttpClient.Builder()`, justo antes de `.build()`:

`datos/ApiClient.kt`

```
.callTimeout(30, TimeUnit.SECONDS)
.addInterceptor { chain ->
    val url = chain.request().url
    val nueva = url.newBuilder()
        .encodedPath(url.encodedPath.lowercase().replace("%20", "-"))   // "Mr%20Mime" → "mr-mime"
        .build()
    chain.proceed(chain.request().newBuilder().url(nueva).build())
}
.build()
```

`%20` es como se escribe un espacio dentro de una URL.

Resumen: lo que aprendiste

1. **URL base** (`ApiClient`) + **ruta** (`@GET`) = la dirección completa de la API.
2. `@Query` agrega `?x=valor` y `@Path` rellena un `{hueco}` en la ruta.
3. Los **DTOs** copian la forma del JSON: objeto → clase, lista → `List`, puede faltar → `?`.
4. El **mapper** traduce los datos de la API a las entidades de tu app. Por eso la pantalla no tuvo que cambiar.

5. Si mantienes los **mismos nombres** (`obtenerPronostico` , `RespuestaClimaDto` , `aEntidad`), el resto del código no se entera del cambio.